23CSE111 OBJECT ORIENTED PROGRAMMING

S2 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

CASE STUDY

PHASE – I (Analysis)

Problem Identification & Class Mapping (CO1)

Group No : C3

Team Members :

Roll No	Student Name
AM.SC.U4CSE25268	Sriha Gara
AM.SC.U4CSE25250	Chaturya Singuru
AM.SC.U4CSE25225	K. V. Pavithra
AM.SC.U4CSE25257	Harshini Vedulla

1. Problem Title : Bank Management System [BMS]

2. Problem Description : A customer named Ravi visits a small digital banking kiosk. Instead of waiting in line, he logs into the Bank Management System using his account ID and passcode. The Login class verifies his credentials by retrieving his account from the database file. Once authenticated, Ravi sees a simple menu: Deposit, Withdraw, Check Balance.

He first deposits ₹5,000. The Deposit transaction object updates his Account balance and immediately persists the change using the Database class. A confirmation message appears. Later, he attempts to withdraw ₹20,000. The Withdraw class checks his balance. Since funds are insufficient, the system politely rejects the request, preventing overdraft.

He then withdraws a smaller amount successfully. Each action updates the stored file, ensuring data integrity. At the end, Ravi checks his balance and logs out. The system ensures secure, consistent, and error-free banking without human calculation.

3. Identified Entities (Classes) :

Class Name	Description
Bank	Manages all banking operations and maintains the list of accounts.
Customer	Represents the Bank Customer actor who performs operations.
BankEmployee	Represents the Bank Staff actor.
Account	Represents the bank account of the customer.
SavingsAccount	It is an inherited class from the Account class used to represent the details related to savings account
CurrentAccount	It is an inherited class from the Account class used to represent the details related to current account
Transaction	Handles deposit and withdrawal operations.
Authentication	Authentication class verifies login credentials by reading customer data from file.

4. Class Attributes and Behaviours :

Class Name	Attributes	Methods(behaviours)
Bank	-bankName : String - accounts : List<Account> -customers : List<Customer>	+addCustomer(customer : Customer) : void +createAccount(customer : Customer, type : int) : Account +findAccount(accountNumber : int) : Account +deposit(accountNumber : int, amount : double) : void

	-transactions : List<Transaction>	+withdraw(accountNumber : int, amount : double) : void +getBalance(accountNumber : int) : double +recordTransaction(transaction : Transaction) : void +getCustomerDetails(customerID : int) : Customer -saveCustomerToFile(customer : Customer) : void -saveTransactionToFile(transaction : Transaction) : void
Customer	-customerID : int -name : String -pin : String -phoneNumber : String	+login(userID : String, pin : String) : boolean +logout() : void +getCustomerID() : int +getCustomerName() : String +getCustomerPhoneNumber() : String
BankEmployee	-employeeID : int -name : String -role : String	+createAccount(bank : Bank, customer : Customer, type : int) : Account +updateAccount(account : Account) : void +viewCustomerDetails(bank : Bank, customerID : int) : Customer
Account	-accountNumber : int -balance : double -accountType : String	+deposit(amount : double) : void +withdraw(amount : double) : void +setBalance(balance : double) : void +getBalance() : double +getAccountNumber() : int
SavingsAccount	-interestRate : double	+calculateInterest() : double
CurrentAccount	-overdraftLimit : double	+checkOverdraft(amount : double) : Boolean +withdraw(amount : double) : void
Transaction	-transactionID : int -amount : double -transactionType : String -date : Date	+ recordTransaction() : void +generateReceipt() : String +getTransactionID() : int +getTransactionType() : String +getTransactionAmount() : double

Authentication	-userID : int -password : String	+verifyCredentials(userID : String, password : String) : boolean